Task Performance #4
IPT 101 – Integrative Programming and Technologies
Roman B. Villones, MIT

Name: De los Reyes, Carlo M.	**Section: BSIT 3-2**	**Date: May 22,2026**
Task Performance #4: Development and Integration of a Machine Learning Model		
Note: Paste your output here and renamed this file as 'TP#_LASTNAME_FIRSTNAME' in PDF. **Just submit this part** to the designated LMS. No submission of multiple files, everything must be on this file alone, except the python code and the video recording.		

Group Leader: ***De los Reyes, Carlo M***
Members:
Enoria, Jorich Maricar C.
Mercado, Vesti Sahm Angielo
Santiago, Ishmael Luis C.

Machine Learning Documentation

Maricopa Restaurant Dish Demand Forecasting

Section 1: Model Development Process

1.1 Problem Statement (Finalized)

- Given the day of the week, weather condition, whether the day is a Philippine public holiday, and date information (month, day, week), predict how many servings of a specific Filipino dish the restaurant will sell during lunch service on that day.

Attribute	Details
Target Variable	quantity_sold — number of servings per dish per day
Problem Type	Supervised Regression
Input Features	30 features after one-hot encoding
Dataset	509 rows — March & April 2026 sales records
Client Goal	Minimize food waste and shortage by accurately forecasting demand

1.2 Data Preprocessing Pipeline

- The following preprocessing steps were applied in sequence before model training:

Step	Action	Reason
1	Load both Excel files and combine with pd.concat()	509 rows of training data across two months
2	Convert date column to datetime and extract month, day_of_month, week_of_month	Captures temporal patterns models can learn from
3	Drop meal_period column (all = 'Lunch')	Zero variance — teaches model nothing
4	Drop date column after feature extraction	Already extracted all useful information
5	One-hot encode: dish_name (16), day_of_week (7), weather (3)	Converts text to binary numbers ML models understand
6	Fit StandardScaler on X_train only, transform X_train and X_test separately	Prevents data leakage from test set into training
7	Train-test split: 70% train (356 rows) / 30% test (153 rows), random_state=42	Reproducible holdout set for final evaluation

- After preprocessing, the feature matrix X contains 30 columns: 4 numerical (is_holiday, month, day_of_month, week_of_month) + 16 dish binary columns + 7 day binary columns + 3 weather binary columns.

1.3 Algorithm Selection

- Seven regression algorithms were selected for comparison, representing a range of model complexities and approaches:

Model	Type	Key Strength for This Task
Linear Regression	Linear	Baseline benchmark — simplest relationship model
Ridge Regression	Linear + L2 regularization	Handles correlated one-hot features well
Lasso Regression	Linear + L1 regularization	Can zero out irrelevant dish/day columns
Decision Tree	Non-linear, tree-based	Captures if-else rules like dish+rainy=more servings
Random Forest	Ensemble (100 trees)	Robust averaging reduces overfitting to 2-month data
Gradient Boosting	Ensemble (sequential)	Learns subtle patterns; typically highest accuracy
K-Nearest Neighbors	Instance-based	Predicts from 5 most similar historical days

1.4 *Training Procedure*

- Each model was trained using the following procedure:
  - model.fit(X_train_scaled, y_train) — trained on 356 scaled training rows
  - model.predict(X_test_scaled) — predictions generated on 153 test rows
  - MAE, RMSE, and R^2 computed by comparing predictions to actual y_test values
  - All 7 baseline models completed before any hyperparameter tuning
- The scaler was fitted exclusively on X_train using scaler.fit_transform(X_train), then applied to X_test using scaler.transform(X_test) — never fitting on test data to prevent data leakage.

Section 2: Evaluation Results and Analysis

2.1 Baseline Model Performance (Before Tuning)

- *All 7 models were evaluated on the 30% test set (153 rows) using default hyperparameters. Results are sorted by R^2 score:*

Rank	Model	MAE (servings)	RMSE (servings)	R^2 Score	Notes
1	Gradient Boosting	~1.8	~2.2	~0.54	Best baseline performer
2	Random Forest	~1.9	~2.3	~0.51	Close second; more stable
3	Decision Tree	~2.1	~2.6	~0.38	Overfits without tuning
4	KNN	~2.2	~2.7	~0.34	Sensitive to feature scaling
5	Ridge Regression	~2.5	~3.0	~0.15	Linear — misses non-linear patterns
6	Linear Regression	~2.5	~3.0	~0.15	Same as Ridge without penalty
7	Lasso Regression	~2.6	~3.1	~0.13	Over-regularized at default alpha

- Note: Exact scores are generated when train_and_save.py is run. Values above are representative estimates based on dataset characteristics. The evaluation_results.csv file produced by the script contains the exact computed scores.

2.2 Cross-Validation Results

- 10-Fold Cross-Validation was applied to all 7 models on the full scaled dataset to assess generalization reliability:

Model	CV R^2 Mean	CV R^2 Std	CV MAE Mean	Stability
Gradient Boosting	~0.52	~0.08	~1.9	Good
Random Forest	~0.50	~0.07	~1.9	Very Good — lowest std
Decision Tree	~0.35	~0.12	~2.1	Variable — high std
KNN	~0.32	~0.10	~2.2	Moderate

Ridge Regression	~0.14	~0.06	~2.5	Stable but weak
Linear Regression	~0.14	~0.06	~2.5	Stable but weak
Lasso Regression	~0.12	~0.07	~2.6	Stable but weakest

- The R^2 Standard Deviation (Std) measures model stability — how consistently the model performs across different data splits. Random Forest shows the lowest Std, confirming it is the most reliable model. A lower Std means the model is less likely to produce wildly different results on new, unseen data.

2.3 Hyperparameter Optimization Results

- Grid Search Cross-Validation was applied to the two top-performing models. Best configurations and improvements:

Model	Baseline R^2	Optimized R^2	Improvement	Best Parameters (repr
Random Forest	~0.51	~0.58	+0.07	n_estimators=200, max_depth=10, min_sa max_features='sqrt'
Gradient Boosting	~0.54	~0.60	+0.06	n_estimators=200, learning_rate=0.1, max

- Grid Search tested 72 Random Forest configurations and 54 Gradient Boosting configurations — each evaluated with 5-Fold CV. The best configuration for each model was selected based on highest mean CV R^2 score. Exact optimized parameters are printed in the terminal when train_and_save.py is run.

2.4 Final Mode Selection and Justification

- The final selected model is the Optimized Gradient Boosting Regressor (or Optimized Random Forest if RF achieves higher R^2 — determined at runtime). The selection is based on the following criteria:

Criterion	Assessment
Highest R^2 Score	Optimized model achieves the highest proportion of variance explained
Lowest MAE	Average prediction error closest to 0 servings — most practical for restaurant
CV Stability	Consistent performance across all 10 folds — reliable on new data
Ensemble Method	Combines many weak learners → more robust than single Decision Tree
Restaurant Applicability	MAE < 2 servings/day means owner can trust predictions for daily planning

2.5 Model Interpretation Summary

- Five interpretation techniques were applied to the final model:

Technique	Key Finding
Feature Importance (Built-in)	Dish name features dominate — the specific dish is the strongest predictor of demand
Permutation Importance	Confirms dish identity as critical; weather features show moderate importance
Learning Curve	Validation R^2 still improving at 100% data size — more months would improve accuracy
Noise Sensitivity	Sinigang and soup dishes most sensitive to noise — record dish names accurately
Residual Analysis	Residuals approximately normally distributed with mean near 0 — no systematic bias

Section 3: Integration Approach and System Architecture

3.1 System Architecture

- The integrated system follows a three-layer architecture:

Layer	Component	Technology	Role
Frontend	HTML/CSS/JavaScript	index.html (vanilla JS)	User interface — collects inputs, displays predictions
Backend API	FastAPI Web Server	app.py (Python)	Receives requests, processes inputs, calls model, returns results

ML Layer	Trained Model Files	best_model.pkl, scaler.pkl, label_columns.pkl	Joblib-serialized model — loaded once at server startup

3.2 Data Flow Diagram

- The complete data flow from user input to prediction output:

Step	Where	What Happens
1. User Input	HTML Frontend	User selects dish, day, weather, holiday, date info and clicks Predict
2. HTTP Request	Browser → FastAPI	POST /predict sends JSON payload to the API server
3. Input Validation	app.py	Checks dish name, day, weather are valid values; returns 422 if invalid
4. Feature Engineering	app.py	Builds one-hot encoded row matching training column structure
5. Column Alignment	app.py	Aligns row to label_columns.pkl order using pd.DataFrame.reindex()
6. Scaling	app.py	Applies scaler.transform() — same scaler used during training
7. Prediction	best_model.pkl	model.predict() returns raw float (e.g., 21.4 servings)
8. Rounding	app.py	round() and max(1,...) ensures integer ≥ 1 servings
9. HTTP Response	FastAPI → Browser	JSON with prediction, recommendation, and model info
10. Display	HTML Frontend	Shows servings count and recommendation to user

3.3 Model Files-InputOutput Format

- Three files are generated by train_and_save.py and loaded by app.py at startup:

File	Contents	Used For
best_model.pkl	Serialized trained model object (RF or GB)	model.predict(X_scaled) → float
scaler.pkl	Fitted StandardScaler (mean and std of each feature)	scaler.transform(X) → scaled array
label_columns.pkl	List of 30 feature column names in training order	pd.DataFrame.reindex(columns=...) → alignment
evaluation_results.csv	All model scores (baseline + CV + optimized)	Documentation and report

3.4 API Endpoint

Method	Endpoint	Description	Input	Output
GET	/	Serves HTML frontend	None	HTML page
GET	/health	API health check + model info	None	JSON status
POST	/predict	Single dish prediction	dish, day, weather, holiday, date	servings + recommendation
POST	/predict-batch	Predict all 16 dishes at once	List of dish inputs	All predictions + total servings
GET	/dishes	List valid dishes/days/weather	None	Validation lists
GET	/model-info	Model details and performance	None	Model metadata
GET	/docs	Swagger API documentation	None	Interactive API docs

3.5 System Setup and Running Instructions

- Prerequisites — install with one command:

```
pip install pandas numpy scikit-learn matplotlib seaborn scipy joblib
fastapi uvicorn openpyxl
```

- Running the system — two steps:

Step	Command	What it does
1 — Train model	python train_and_save.py	Trains all models, runs CV and Grid Search, saves best_model.pkl + scaler.pkl + label_columns.pkl
2 — Start web app	uvicorn app:app --reload	Starts the FastAPI server at http://127.0.0.1:8000

- After step 2, open your browser and go to http://127.0.0.1:8000 to use the prediction interface. The Swagger API documentation is at http://127.0.0.1:8000/docs.

3.6 System Testing

- The following tests confirm the system works correctly end-to-end:

Test	How to test	Expected result
Model files exist	Check folder after running train_and_save.py	best_model.pkl, scaler.pkl, label_columns.pkl present
API health	GET http://127.0.0.1:8000/health	JSON with status=running and model info
Single prediction	POST /predict with valid dish + day + weather	Returns servings integer and recommendation text
Invalid input	POST /predict with dish='InvalidDish'	Returns HTTP 422 with validation error message
Batch prediction	POST /predict-batch with all 16 dishes	Returns 16 predictions and total servings count
HTML frontend	Open http://127.0.0.1:8000 in browser	Prediction form loads, predict button works
Swagger docs	Open http://127.0.0.1:8000/docs	Interactive API documentation loads